

Part A - Advanced SQL (10 Marks)

Schema:

Students(student_id, name, dob, department_id)

Departments(department_id, department_name)

Courses(course_id, course_name, department_id)

Enrollments(enrollment_id, student_id, course_id, grade)

11) Create a view named TopStudents that shows student_id, name, and average_grade for students whose average grade is greater than 3.5.

Solution (SQL):

```
CREATE VIEW TopStudents AS
SELECT s.student_id, s.name, AVG(e.grade) AS average_grade
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
HAVING AVG(e.grade) > 3.5;
```

12) Write a stored procedure to increase the grade of all students in a given course by 0.5 (maximum grade is 4.0).

Solution (MySQL-style SQL):

```
DELIMITER //
CREATE PROCEDURE IncreaseGradesForCourse(IN in_course_id INT)
BEGIN
    UPDATE Enrollments
    SET grade = LEAST(grade + 0.5, 4.0)
    WHERE course_id = in_course_id;
END;
//
DELIMITER ;
```

Explanation: LEAST ensures the grade does not exceed 4.0. This updates the enrollments (grades) for the specified course_id.

13) Using a correlated subquery, list the students whose grade in any course is the highest grade for that course.

Solution (SQL):

```
SELECT DISTINCT s.student_id, s.name, e.course_id, e.grade
FROM Enrollments e
JOIN Students s ON e.student_id = s.student_id
WHERE e.grade = (
    SELECT MAX(e2.grade)
    FROM Enrollments e2
    WHERE e2.course_id = e.course_id
);
```

Explanation: For each enrollment row 'e', the subquery finds the maximum grade for that course. If e.grade equals that maximum, the student is included.